

JAVA 8 MASTER HANDBOOK

Lecture 1 – Java 8 Introduction

Part 3 – Java 8 Features Overview

README.md (Section 2)

Chapter 5 - Method References (Overview)

Ab ek interesting question.

Suppose tum Lambda Expression likh rahe ho.

```
list.forEach(name -> System.out.println(name));
```

Observe carefully.

Lambda ke andar koi naya logic nahi likha gaya.

Ye sirf

```
System.out.println(name);
```

ko call kar raha hai.

Question.

Agar Lambda sirf kisi existing method ko call kar raha hai,

to Lambda likhne ki bhi kya zarurat hai?

Isi problem ko solve karne ke liye

Java 8 ne introduce kiya

Method References

Java 8 Solution

```
list.forEach(System.out::println);
```

Observe.

Lambda gayab ho gaya.

Sirf method ka reference pass kar diya.

Definition

Method Reference ek concise syntax hai

jo existing method ko directly refer karta hai

jab Lambda sirf wahi method call kar raha ho.

Purpose

Method References ka main objective hai

- Cleaner Code
 - Better Readability
 - Less Boilerplate
 - Existing Methods ka Reuse
-

Important Note

Har Lambda ko Method Reference me convert nahi kiya ja sakta.

Sirf wahi Lambdas convert hote hain

jo directly kisi existing method ko call karte hain.

Ye detail hum Method Reference lecture me deeply padhenge.

Real World Analogy

Suppose tumhare ghar me TV ka Remote hai.

Remote pe

Volume Up

already bana hua hai.

Tum har baar

Button Press

↓

Increase Volume

↓

Update Display

nahi likhte.

Sirf

Volume +

button dabate ho.

Exactly

Method Reference bhi

existing functionality ko

directly refer karta hai.

Chapter 6 - Stream API (Overview)

Java 8 ka

sabse famous feature.

Interview me

sabse zyada pucha jane wala feature.

Definition

Stream API

Collections ko

Declarative style me process karne ka framework hai.

Stream Pipeline

Collection

↓

Stream

↓

Filter

↓

Map

↓

Sort

↓

Collect

↓

Result

Ye diagram yaad kar lena.

Poora Stream API isi pipeline ke around hai.

Example

```
employees.stream()  
    .filter(e -> e.getSalary() > 50000)  
    .collect(Collectors.toList());
```

Yahan

Collection

↓

Stream

↓

Filter

↓

Collect

↓

List

Stream API ka Purpose

Java developers ne observe kiya

ki collections process karne ke liye

baar baar

```
for(...)
```

likhna padta tha.

Java 8 ne kaha

Developer

sirf

bataye

ki

kya chahiye.

Execution

Stream handle karega.

Interview Point

Question

Stream data store karti hai?

Answer

No.

Streams

data store nahi karti.

Streams

sirf data ko process karti hain.

Ye question bahut interviews me pucha jata hai.

Stream API Benefits

- Less Code
 - Better Readability
 - Declarative Style
 - Internal Iteration
 - Parallel Processing Support
 - Functional Programming Support
-

Chapter 7 - Default Methods

Ab ek bahut important feature.

Question.

Java 7 me

Interface

ke andar implementation kyun nahi hoti thi?

Example

```
interface Animal{  
    void sound();  
}
```

100 classes

is interface ko implement kar rahi hain.

Ab company ne decide kiya

interface me

ek aur method add karna hai.

```
void eat();
```

Kya hoga?

Answer

Saari 100 classes

compile error dene lagenge.

Kyunki

ab unhe

eat()

implement karna padega.

Ye bahut badi problem thi.

Java 8 Solution

```
interface Animal{  
  
    void sound();  
  
    default void eat(){  
  
        System.out.println("Eating");  
  
    }  
  
}
```

Ab

existing classes

break nahi honggi.

Purpose

Default Methods ka main purpose hai

Backward Compatibility

Old code ko break kiye bina

Interface evolve karna.

Interview Question

Why Default Methods were introduced?

Answer

Existing interfaces me

new functionality add karne ke liye

without breaking

existing implementation classes.

Chapter 8 - Static Methods in Interfaces

Java 8 se pehle

Interfaces ke andar

Static Methods

allowed nahi the.

Java 8 ke baad

ye possible ho gaya.

Example

```
interface MathUtil{  
  
    static int square(int x){  
  
        return x*x;  
  
    }  
  
}
```

Call

```
MathUtil.square(5);
```

Observe.

Object banane ki zarurat nahi.

Purpose

Utility methods ko

usi Interface ke saath

group karke rakhna.

Isse code

zyada organized ho jata hai.

Default Method vs Static Method

Default Method	Static Method
Object se call hota hai	Interface Name se call hota hai
Override ho sakta hai	Override nahi hota
Inherited hota hai	Inherited nahi hota

Ye comparison interview me kaafi baar pucha jata hai.

Chapter 9 - Optional

Java developers ne

ek aur bahut common problem notice ki.

Problem

```
Employee employee = findEmployee();  
  
employee.getName();
```

Question.

Agar

findEmployee()

null return kare

to kya hoga?

Answer

```
NullPointerException
```

Aur Java developers jaante the

ki

NPE

Java applications me

sabse common runtime exceptions me se ek hai.

Java 8 Solution

Optional<Employee>

Definition

Optional

ek container object hai

jo represent karta hai

ki value

present bhi ho sakti hai

aur absent bhi.

Purpose

- Null Handling Improve karna
 - Code Readability Improve karna
 - API Design Better banana
 - Null ko Explicit banana
-

Important Note

Interview me

kabhi mat bolna

Optional NullPointerException ko completely remove kar deta hai.

Ye galat hai.

Correct answer

Optional

null ko safely represent karne ka

ek better mechanism provide karta hai.

Chapter 10 - Date & Time API

Java ke old Date APIs

bahut problematic the.

Old Classes

- Date
 - Calendar
 - SimpleDateFormat
-

Problems

- Mutable
 - Thread Unsafe (`SimpleDateFormat`)
 - Difficult API
 - Confusing Methods
-

Java 8 Solution

New Classes

- LocalDate
 - LocalTime
 - LocalDateTime
 - Instant
 - Duration
 - Period
 - ZonedDateTime
-

Benefits

- Immutable
- Thread Safe
- Easy API
- Better Design

- Better Readability
-

Example

```
LocalDate today = LocalDate.now();  
  
System.out.println(today);
```

Ye code

old Date API se

kaafi readable hai.

Interview Point

Question

Why Java 8 introduced New Date-Time API?

Answer

Old Date API

Mutable thi.

Thread Safe nahi thi.

Aur difficult thi.

Java 8 ne

immutable

thread-safe

aur modern API provide ki.

Quick Revision Box

Method Reference

↓

Cleaner Lambda

↓

Stream API

↓

Collection Processing

↓

Default Method

↓

Backward Compatibility

↓

Static Interface Method

↓

Utility Methods

↓

Optional

↓

Null Handling

↓

Date-Time API

↓

Modern Date Handling